

ChatGPT ▾

[Log in](#)[Sign up for free](#)

I'm hoping to code a nonogram game. Help me plan it (not code it)

A good nonogram project is large enough to be interesting but small enough that you can build it in layers. The biggest mistake is trying to build everything at once. Instead, think of it as several independent systems that gradually come together.

Here's one way to structure the project.

1. Define the game's scope

Start by deciding what your first playable version ("Version 1") will include.

For example:

- Fixed puzzle sizes (5×5 and 10×10)
- Mouse controls
- Manual solving only
- Puzzle completion detection
- Restart button
- No puzzle editor
- No hints
- No solver

Everything else can come later.

2. Identify the major systems

Think of the game as several modules.

Puzzle Model

Responsible for:

- Puzzle dimensions
- Solution grid
- Row clues
- Column clues

It should know nothing about graphics.

Player Board

Stores the player's current progress.

Each cell might have three states:

- Unknown
- Filled
- Marked empty (X)

This is separate from the solution.

Puzzle Generator / Loader

Decide where puzzles come from.

Options:

- Hardcoded puzzles
- JSON files
- User-created puzzles
- Random generator (much harder)

Most games begin with a library of predefined puzzles.

Clue Generator

Given a completed solution grid, produce:

■ ■ □ ■ ■

becomes

2 2

This is useful if you ever add a puzzle editor.

Renderer

Responsible only for displaying:

- Grid
- Cells
- Clues
- Cursor
- Selection
- Win animation

It shouldn't contain game logic.

Input Controller

Converts player actions into board changes.

Examples:

- Left click = fill
- Right click = X
- Drag to paint
- Keyboard shortcuts

Rule Checker

Determines:

- Has the puzzle been solved?
- Are there mistakes?
- Is the board complete?

Some games immediately reveal errors.

Others only check at the end.

Make this configurable.

UI

Menus

Settings

Puzzle selection

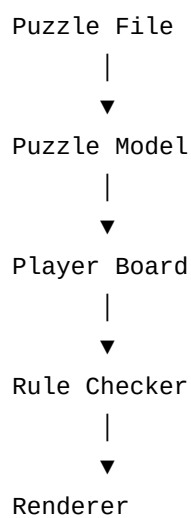
Pause

Timer

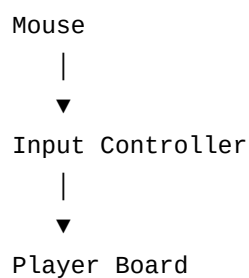
Statistics

3. Decide how data flows

A simple architecture might look like:



Input goes the opposite direction:



Keeping rendering and game logic separate makes the project much easier to maintain.

4. Think about game states

Instead of dozens of boolean flags:

```
isPlaying  
isPaused  
isWin  
isMenu  
isLoading
```

Use one state machine.

For example:

```
Main Menu  
  
Puzzle Select  
  
Playing  
  
Paused  
  
Puzzle Complete  
  
Settings
```

Only one state is active at a time.

5. Plan your data structures

Think about what information you need.

Cell

- Position
- Current state

Puzzle

- Width
- Height
- Solution
- Row clues
- Column clues
- Name
- Difficulty

Player Save

- Elapsed time
- Current board
- Mistakes
- Puzzle completed

6. Decide how puzzles are stored

Example structure:

```
Puzzle
  Name

  Width

  Height

  Solution Grid

  Row Clues

  Column Clues

  Difficulty
```

Even if clues can be generated automatically, storing them may reduce loading time.

7. Plan the gameplay loop

The basic loop is simple:

Load puzzle

↓

Draw board

↓

Wait for input

↓

Update player board

↓

Check completion

↓

Redraw

↓

Repeat

8. Make a feature roadmap

Instead of trying to build everything immediately.

Milestone 1

- Display grid
- Draw clues
- Load puzzle

Milestone 2

- Click cells
- Toggle states
- Restart

Milestone 3

- Win detection
- Victory screen
- Timer

Milestone 4

- Save progress
- Multiple puzzles
- Puzzle selection

Milestone 5

- Puzzle editor
- Auto clue generation

Milestone 6

- Hint system
- Solver
- Undo/Redo

Milestone 7

- Themes
- Animations
- Statistics
- Achievements

9. Consider quality-of-life features

Players often appreciate:

- Undo/Redo
- Drag to fill multiple cells
- Drag to mark multiple Xs
- Zoom for large puzzles

- Auto-cross completed lines
- Highlight current row and column
- Highlight matching clues
- Mistake counter (optional)
- Timer (optional)
- Hint system
- Keyboard controls
- Accessibility options (high contrast, colorblind-friendly cues)

10. Think about extensibility

If you may add features later, avoid tying everything together too tightly.

Potential future additions include:

- Daily puzzles
- Puzzle packs
- User-created puzzles
- Sharing puzzles via import/export
- Multiple color nonograms
- Hexagonal or triangular grids
- Cooperative solving
- Leaderboards
- Touch support

Designing clear boundaries between the puzzle model, player state, rendering, and input will make these additions much easier.

A practical development order

A sequence that minimizes rework is:

1. Display an empty grid.
2. Load and display a puzzle with clues.
3. Allow filling and marking cells.
4. Detect puzzle completion.
5. Add menus and puzzle selection.
6. Implement save/load.
7. Add undo/redo.
8. Add polish (animations, sounds, themes).
9. Build advanced features like hints or a puzzle editor.

This order gives you a playable game early and lets you iterate from a solid foundation rather than trying to solve rendering, input, puzzle logic, and UI all at once.

